

prog_datasci_8_vis_exercicisResolts

December 28, 2020

1 Programació per a *Data Science*

1.1 Unitat 8: Visualització de dades en Python - Exercicis Resolts

En aquest Notebook hi trobareu un conjunt d'exercicis per practicar. Aquests exercicis no puntuen per a la PAC, però us recomanem que els intenteu resoldre com a part del procés d'aprenentatge. Trobareu exemples de possibles solucions als exercicis al propi notebook, però és important que intenteu resoldre'ls vosaltres abans de consultar les solucions. Les solucions us han de permetre validar les vostres respostes, així com veure alternatives de resolució de les activitats. També us animem a preguntar qualsevol dubte que us sorgeixi sobre la resolució de les activitats per practicar al fòrum de l'aula.

1.2 Preguntes i exercicis per a practicar

1.2.1 Exercici 1

Pokemon és una saga RPG molt famosa a nivell mundial. La missió d'aquests jocs és capturar i entrenar els pokemon, unes criatures que habiten arreu del continent, per fer-se amb el títol de mestre de la lliga Pokemon. Els Pokemons són criatures molt diverses, hi ha molts tipus diferents i alguns d'ells poden evolucionar.

En aquest exercici explorarem els Pokemons de la primera generació amb el dataset `pokemon.csv`.

1. Quants Pokemons llegendaris hi ha? Quants Pokemons hi ha de cada tipus d'evolució (variable *Stage*)?
2. Quin és el tipus més freqüent? I el menys?
3. Un nou entrenador ha d'escollir entre 3 Pokemons inicials (Bulbasaur, Charmander i Squirtle) i ens ha demanat que l'ajudem a decidir basant-nos en les estadístiques dels pokemons d'aquests tres tipus (Grass, Fire i Water). Si ens centrem només en les característiques d'atac i defensa, quin tipus li hauríem de recomanar?

Representa les respostes gràficament.

Pista: Per la 3a pregunta, podeu utilitzar la funció `jointplot` que hem vist al Notebook d'explicació. Penseu en quin tipus de gràfica, d'entre les que ofereix `jointplot`, s'ajusta als requeriments de l'enunciat.

```
[1]: # Resposta
```

1.2.2 Exercici 2

Carregueu el conjunt de dades `pulitzer-circulation-data.csv` en un dataframe de Pandas. La font original d'aquest conjunt de dades és el repositori de dades de [FiveThirtyEight](#).

Genereu un diagrama de dispersió que permeti visualitzar les mostres del conjunt de dades de Pulitzer segons les variables `Daily Circulation 2004` i `Daily Circulation 2013`. Incloeu una recta amb l'ajust lineal entre les dues variables. Pel que fa als detalls de visualització, limiteu la visualització de tots dos eixos a l'interval (0, 1000000), feu servir l'estil `whitegrid` de Seaborn, i dibuixeu els punts i la línia en color verd.

Pista: podeu utilitzar la funció `jointplot` que hem vist al Notebook d'explicació. Penseu en quin tipus de gràfica, [d'entre les que ofereix jointplot](#), s'ajusta als requeriments de l'enunciat.

```
[2]: # Resposta
```

1.2.3 Exercici 3

Joc de Trons és una serie de televisió basada en la famosa saga *Una Cançó de Foc i Gel* de George RR Martin. Aquesta serie és coneguda pels complicats escenaris polítics i bèlics, així com les nombroses morts de personatges.

En aquest exercici treballem amb el dataset `battles.csv` que ens dona informació sobre les batalles que han tingut lloc al llarg de *Joc de Trons*.

Genereu un diagrama de punts (*scatter*) que permeti comparar la mida (`attacker_size` vs `defender_size`) dels dos principals exèrcits involucrats en cada batalla. Feu servir la columna `attacker_outcome` per mostrar amb diferents colors el resultat de la batalla per l'atacant.

Podeu utilitzar la funció `lplot` de Seaborn per generar la gràfica. Penseu en quins atributs ens permeten crear aquest tipus de visualització.

Veient la gràfica, podem afirmar que la mida de l'exèrcit és determinant en l'*outcome* de la batalla?

Nota: feu servir l'estil `whitegrid` de Seaborn.

```
[3]: # Resposta
```

1.2.4 Exercici 4

En els regnes de Ponent, els Grans Mestres consideren que tothom hauria de tenir accés a una biblioteca. Desgraciadament, després d'una gran guerra la xarxa de carreteres que comuniquen aquests regnes ha quedat greument afectada. Els Grans Mestres ens han contractat per resoldre les següents qüestions:

1. Quins regnes han quedat sense poder accedir a una biblioteca?
2. Quin és el nombre mínim de biblioteques que s'haurien de construir per tal de que tothom torni a tenir accés a una biblioteca? (si no podem re-construir cap carretera)
3. Quin és el cost mínim de re-construcció de carreteres per tal de que tothom torni a tenir accés a una biblioteca? (si no podem construir cap biblioteca)
4. Veient la resposta de la pregunta 2 i 3, què seria el més òptim tenint en compte que volem minimitzar els costos?

Per poder respondre a aquestes preguntes, els Grans Mestres ens faciliten la següent informació:

- Disposem d'un graf amb l'estat de la xarxa de carreteres entre els regnes.
- Disposem d'un llistat amb les ciutats que disposen d'una biblioteca.
- Una ciutat es considera que té accés a una biblioteca si existeix una biblioteca en la mateixa ciutat o si els habitants poden viatjar per carretera fins a una altra ciutat amb biblioteca.
- El cost de construir una biblioteca és de 150.000€.
- El cost de re-construir una carretera és de 80.000€.

Nota: per resoldre l'exercici, **heu de generar una visualització del graf que us permeti respondre a les quatre preguntes plantejades**. Després, recordeu respondre a les quatre preguntes!

```
[4]: # Dades disponibles per a l'exercici

import networkx as nx

# Llista de carreteres transitables
carreteres_ok = [(1,3), (3,4), (2, 7), (2, 8), (5, 6), (9, 10)]

# Llista de carreteres tallades
carreteres_ko = [(1, 2), (2, 4), (2, 5), (8, 9)]

# Graf G, que representa la xarxa de carreteres entre les ciutats
G = nx.Graph()

G.add_edges_from(carreteres_ok)
G.add_edges_from(carreteres_ko)

# Llista de ciutats amb biblioteca
ciutats_amb_biblioteca = [3, 6, 9]
```

```
[5]: # Resposta
```

1.2.5 Exercici 5

L'any 2018 a la ciutat de Barcelona s'han produït, de mitjana, 27.22 accidents amb vehicles implicats. És per això que l'ajuntament de Barcelona requereix del nostre servei per construir un mapa **interactiu** que mostri els punts on s'han produït els accidents.

Ens demanen crear un mapa utilitzant la llibreria *geoplotlib* que ens ha de permetre veure la localització de cada accident i el nombre de víctimes implicades. De moment només ens demanen visualitzar els accidents produïts al mes de **juny del 2018** en **caps de setmana** (dissabtes i diumenges).

Nota 1: Podeu obtenir les dades dels accidents gestionats per la Guàrdia Urbana a la ciutat de Barcelona al portal [Open Data BCN](#) i [carregar les dades](#) a partir d'un diccionari o un dataframe.

```
[6]: # Resposta
```

1.2.6 Exercici 6

En aquest exercici utilitzarem el *dataset covid19_US_cases.csv*, que mostra l'evolució dels casos de Covid-19 als diferents estats des de finals de gener del 2020.

1. Quina és la distribució de casos als Estats Units l'últim mes (abril)? **Mostra gràficament els resultats per cada grup (actius, recuperats i morts), però sense separar per estats.**
2. Addicionalment, se'ns demana generar una gràfica que mostri l'evolució dels casos confirmats a l'estat de Nova York, Massachusetts, Washington, California i Texas des de l'inici de la pandèmia. Quina regió és la més afectada?

Nota 1: En l'apartat 1, per sumar tots els casos de US es pot utilitzar la funció `groupby` de `pandas` que hem vist en lliçons anteriors.

Nota 2: La visualització de les dades hauria de ser atractiva i clara per tal de que es puguin treure conclusions d'elles.

[]: `# Resposta`

1.2.7 Exercici 7

A partir de les dades de twitter es poden crear xarxes de relacions entre els seus usuaris. En aquest exercici treballarem amb el conjunt de dades *politics_twitter.csv*, que constitueix una xarxa de les relacions entre diferents polítics de l'estat espanyol generada a partir d'extraccions de twitter.

Cada fila representa un link entre 2 polítics (P1 i P2) i el valor (Weight) indica la força de la connexió, en aquest cas 1 ja que només fa referència a la presència d'aquesta relació.

1. Quantes comunitats es poden detectar al graf?
2. Representa gràficament el graf i pinta de diferent color cada comunitat detectada.

[]: `# Resposta`

1.3 Solucions exercicis per practicar

1.3.1 Exercici 1

Pokemon és una saga RPG molt famosa a nivell mundial. La missió d'aquests jocs és capturar i entrenar els pokemon, unes criatures que habiten arreu del continent, per fer-se amb el títol de mestre de la lliga Pokemon. Els Pokemons són criatures molt diverses, hi ha molts tipus diferents i alguns d'ells poden evolucionar.

En aquest exercici explorarem els Pokemons de la primera generació amb el dataset *pokemon.csv*.

1. Quants Pokemons llegendaris hi ha? Quants Pokemons hi ha de cada tipus d'evolució (variable *Stage*)?
2. Quin és el tipus més freqüent? I el menys?

3. Un nou entrenador ha d'escollir entre 3 Pokemons inicials (Bulbasaur, Charmander i Squirtle) i ens ha demanat que l'ajudem a decidir basant-nos en les estadístiques dels pokemons d'aquests tres tipus (Grass, Fire i Water). Si ens centrem només en les característiques d'atac i defensa, quin tipus li hauríem de recomanar?

Representa les respostes gràficament.

Pista: Per la 3a pregunta, podeu utilitzar la funció `jointplot` que hem vist al Notebook d'explicació. Penseu en quin tipus de gràfica, d'entre les que ofereix `jointplot`, s'ajusta als requeriments de l'enunciat.

```
[7]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Mostrem les gràfiques en el notebook
%matplotlib inline

# Indiquem que volem utilitzar l'estil "whitegrid" de seaborn
sns.set_style("whitegrid")

# Desactivem uns warnings perquè no ens apareguin
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=RuntimeWarning)

# Carreguem les dades del fitxer pokemon.csv en un dataframe
pkmn = pd.read_csv('data/pokemon.csv')

# Visualitzem les primeres files del dataset
pkmn.head()
```

```
[7]:
```

	Number	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	\
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	
2	3	Venusaur	Grass	Poison	525	80	82	83	100	
3	4	Charmander	Fire	NaN	309	39	52	43	60	
4	5	Charmeleon	Fire	NaN	405	58	64	58	80	

	Sp. Def	Speed	Stage	Legendary
0	65	45	1	False
1	80	60	2	False
2	100	80	3	False
3	50	65	1	False
4	65	80	2	False

```
[8]: ##1

#Quants Pokemons llegendaris hi ha?

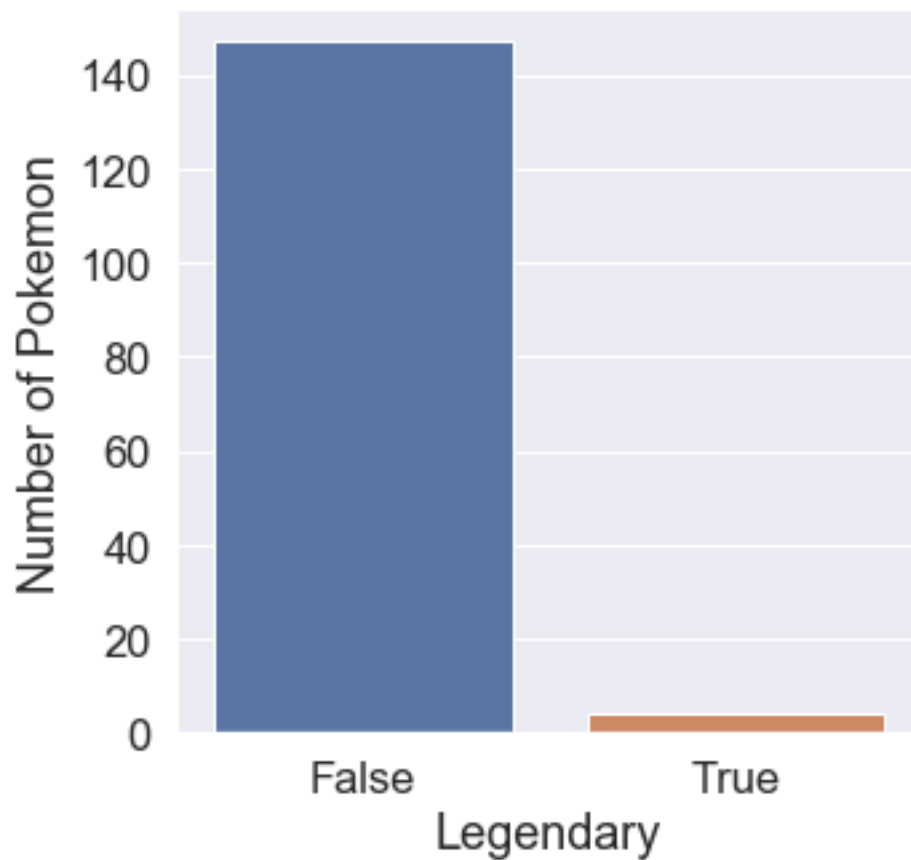
pkmn_no_leg=np.sum(pkmn['Legendary']==0)
pkmn_leg=np.sum(pkmn['Legendary']==1)

print(f"Número de Pokemons no llegendaris: {pkmn_no_leg}")
print(f"Número de Pokemons llegendaris: {pkmn_leg}")

#Grafiquem la resposta
plt.figure(figsize=(5,5))
sns.set(font_scale = 1.5)
legend_ = sns.countplot(x="Legendary", data=pkmn)
legend_ = plt.ylabel('Number of Pokemon')
```

Número de Pokemons no llegendaris: 147

Número de Pokemons llegendaris: 4



```
[9]: #Quants pokemons hi ha de cada evolució?

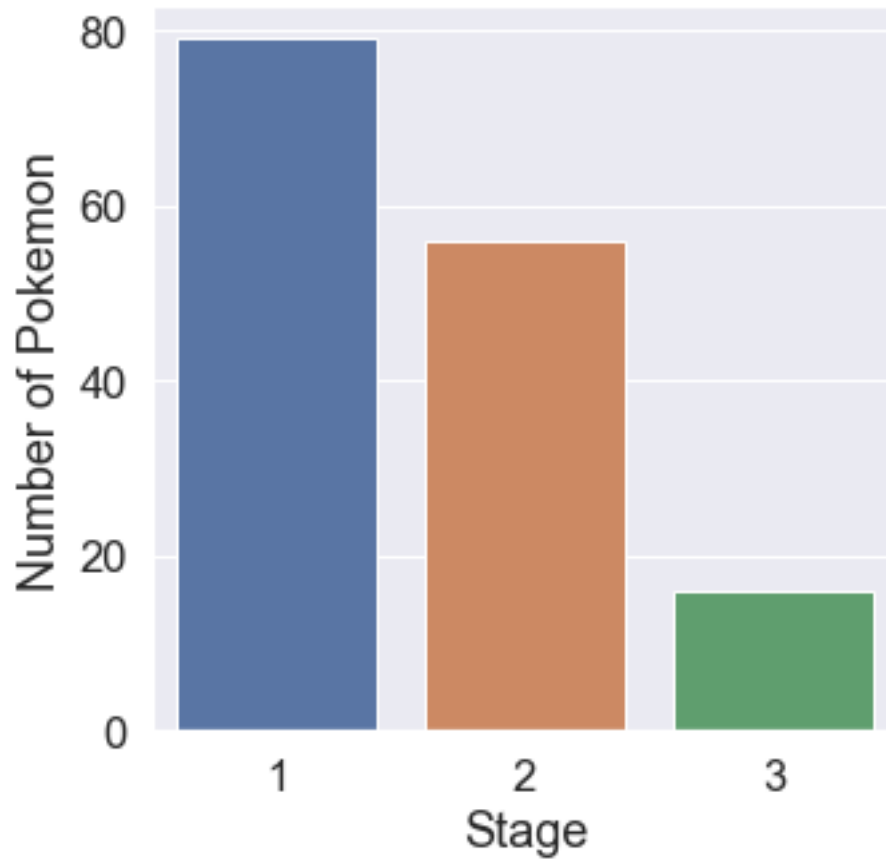
#OPCIÓ A
pkmn_S1=np.sum(pkmn['Stage']==1)
pkmn_S2=np.sum(pkmn['Stage']==2)
pkmn_S3=np.sum(pkmn['Stage']==3)

print(f"Número de Pokemons Stage 1: {pkmn_S1}")
print(f"Número de Pokemons Stage 2: {pkmn_S2}")
print(f"Número de Pokemons Stage 3: {pkmn_S3}")

#OPCIÓ B
pkmn_ = pkmn.groupby('Stage').count().Number

#Grafiquem la resposta
plt.figure(figsize=(5,5))
sns.set(font_scale = 1.5)
stage_ = sns.countplot(x="Stage", data=pkmn)
stage_ = plt.ylabel('Number of Pokemon')
```

```
Número de Pokemons Stage 1: 79
Número de Pokemons Stage 2: 56
Número de Pokemons Stage 3: 16
```



```
[10]: ##2

#Quin és el tipus més freqüent? I el menys?

#Mostrem el número de pokemons per cada tipus
print(pkmn.groupby('Type 1')['Number'].count())

print("\n")

#Mostrem quin és el tipus més i menys frequent

print(f"El tipus més freqüent és el de " + str(pkmn.groupby('Type 1')['Number'].
    ↳count().argmax()) +
      " amb " + str(pkmn.groupby('Type 1')['Number'].count().max()) + "↳
    ↳pokemons")
print(f"El tipus menys freqüent és el de " + str(pkmn.groupby('Type 1')['Number'].count().idxmin()) +
      " amb " + str(pkmn.groupby('Type 1')['Number'].count().min()) + "↳
    ↳pokemons")
```



```

#Indiquem que volem utilitzar l'estil "whitegrid" de seaborn
sns.set_style("whitegrid")

#Grafiquem la resposta
plt.figure(figsize=(10,7))
sns.set(font_scale = 1.5)
type_ = sns.countplot(x="Type 1", data=pkmn)
type_ = plt.xticks(rotation=45)
type_ = plt.ylabel('Number of Pokemon')

```

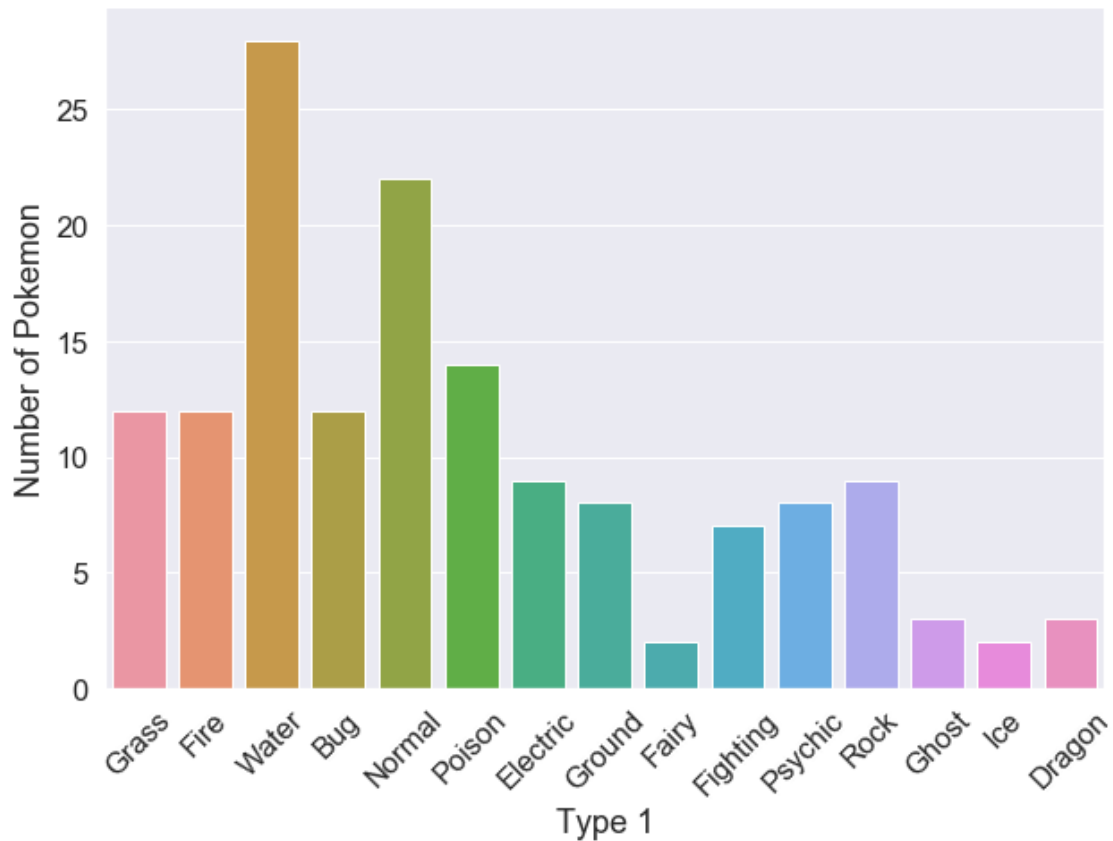
```

Type 1
Bug      12
Dragon   3
Electric 9
Fairy    2
Fighting 7
Fire     12
Ghost    3
Grass    12
Ground   8
Ice      2
Normal   22
Poison   14
Psychic  8
Rock     9
Water    28
Name: Number, dtype: int64

```

El tipus més freqüent és el de 14 amb 28 pokemons

El tipus menys freqüent és el de Fairy amb 2 pokemons



```
[11]: ##3

#Quin pokemon inicial hem d'escollir?

#Seleccionem els pokemons de tipus aigua, foc i planta.
pkmn_inicials_tipus = pkmn[pkmn["Type 1"].isin(["Grass", "Fire", "Water"])]

#Seleccionem les característiques que volem evaluar
pkmn_inicials_tipus_stats=pkmn_inicials_tipus[["Type_
↳1", "HP", "Attack", "Defense"]]

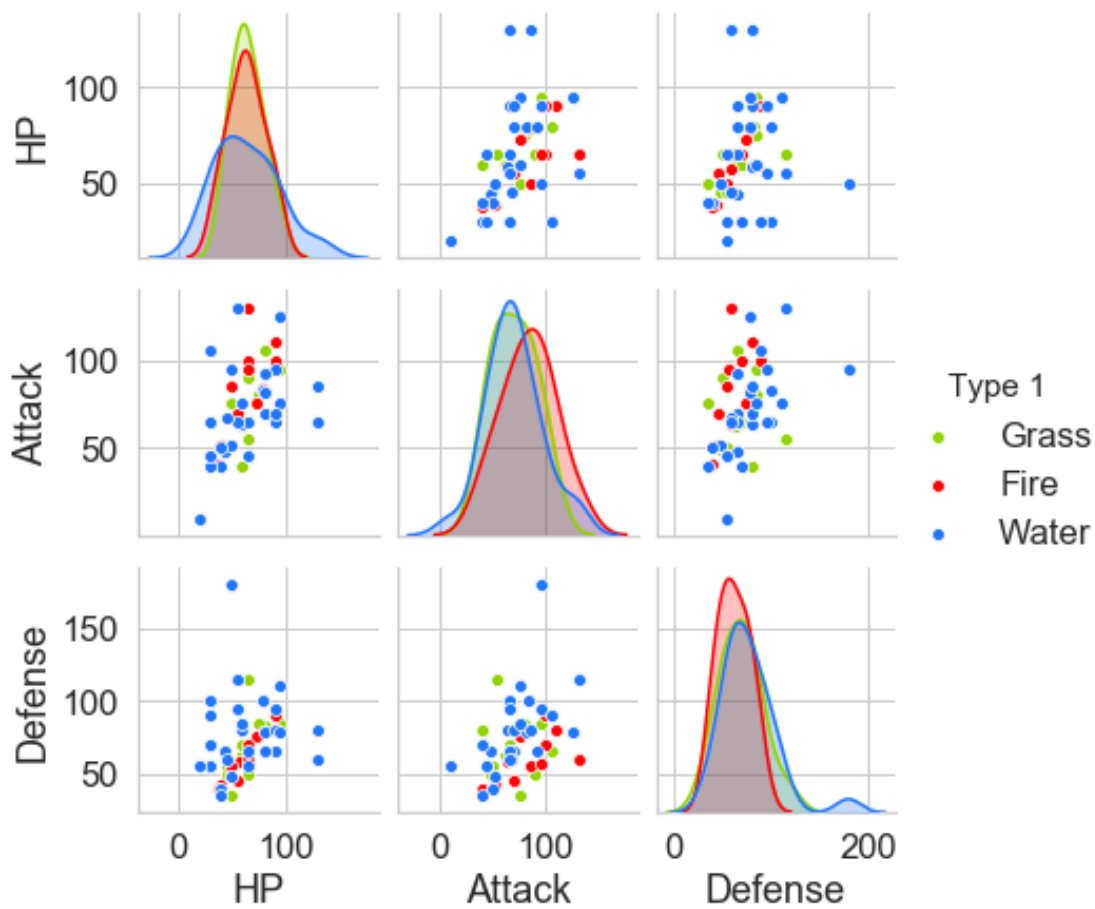
#Especifiquem uns colors concrets pels tipus de pokemon en els plots_
↳(verd=planta, blau=aigua, vermell=foc)
colors_pkm=["#8fd400", "#ff0000", "#1f75fe"]
```

```
[12]: #OPCIÓ A: pairplot

#Indiquem que volem utilitzar l'estil "whitegrid" de seaborn
sns.set_style("whitegrid")
```

```
#Grafiquem la resposta
sns.pairplot(pkmn_inicials_tipus_stats, hue="Type 1",height=2,
→palette=colors_pkm)
```

[12]: <seaborn.axisgrid.PairGrid at 0x1a204861d0>

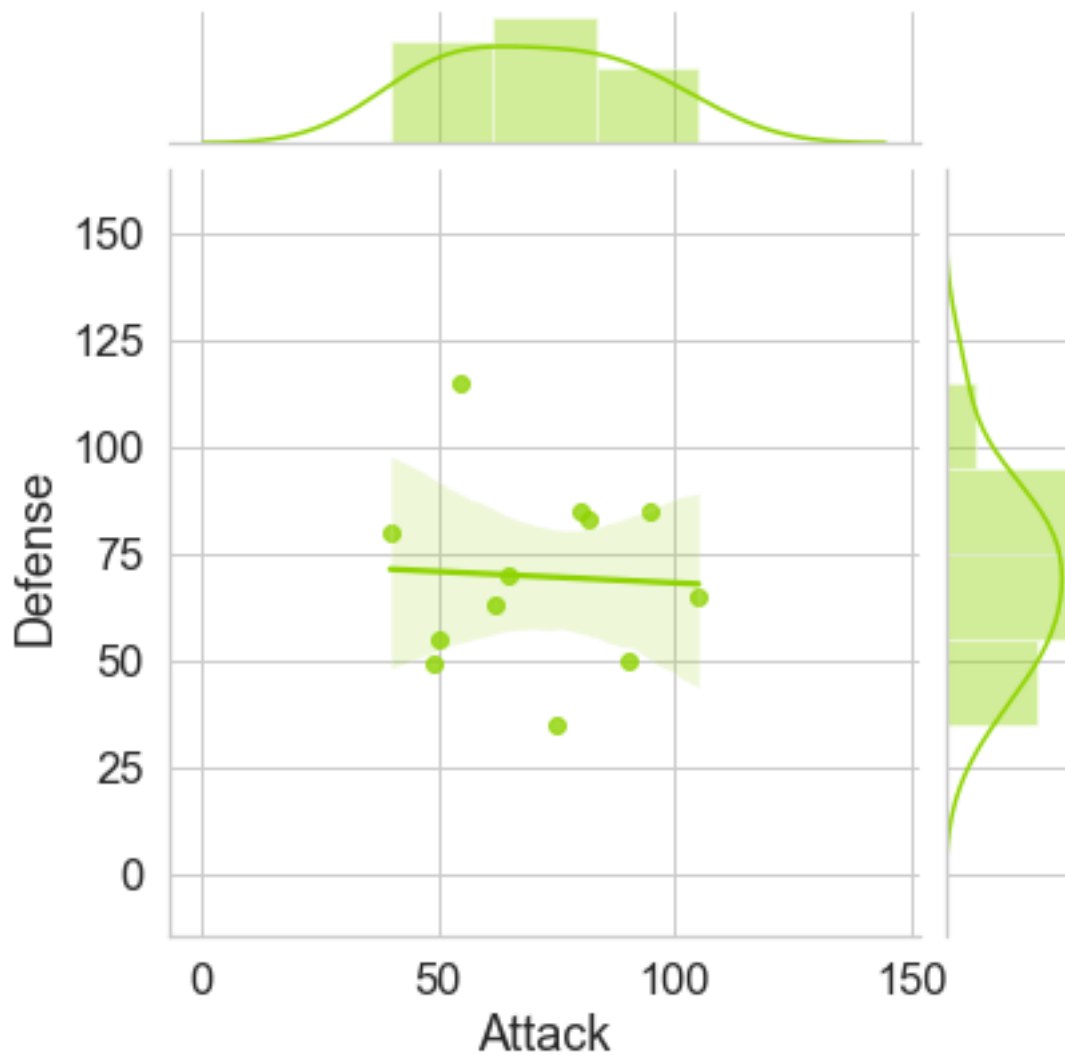


[13]: #OPCIÓ B: jointplot

```
#Indiquem que volem utilitzar l'estil "whitegrid" de seaborn
sns.set_style("whitegrid")

# Joint Distribution Plot
sns.jointplot(x='Attack', y='Defense',
→data=pkmn_inicials_tipus_stats[pkmn_inicials_tipus_stats['Type 1']=='Grass'],
color="#8fd400",kind='reg')
```

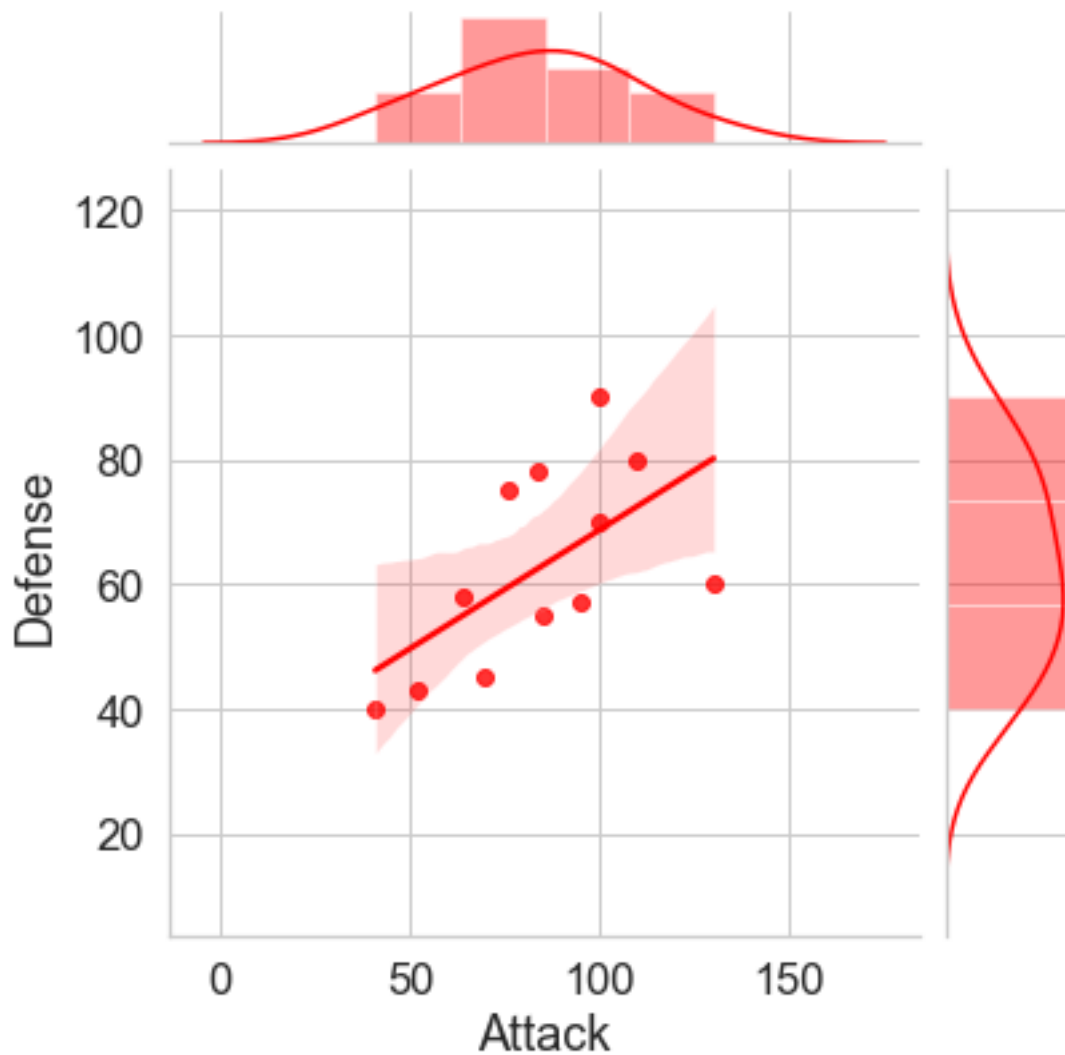
[13]: <seaborn.axisgrid.JointGrid at 0x1a209ba950>



```
[14]: #Indiquem que volem utilitzar l'estil "whitegrid" de seaborn
sns.set_style("whitegrid")

sns.jointplot(x='Attack', y='Defense',
             data=pkmn_inicials_tipus_stats[pkmn_inicials_tipus_stats['Type 1']=='Fire'],
             color="#ff0000",kind='reg')
```

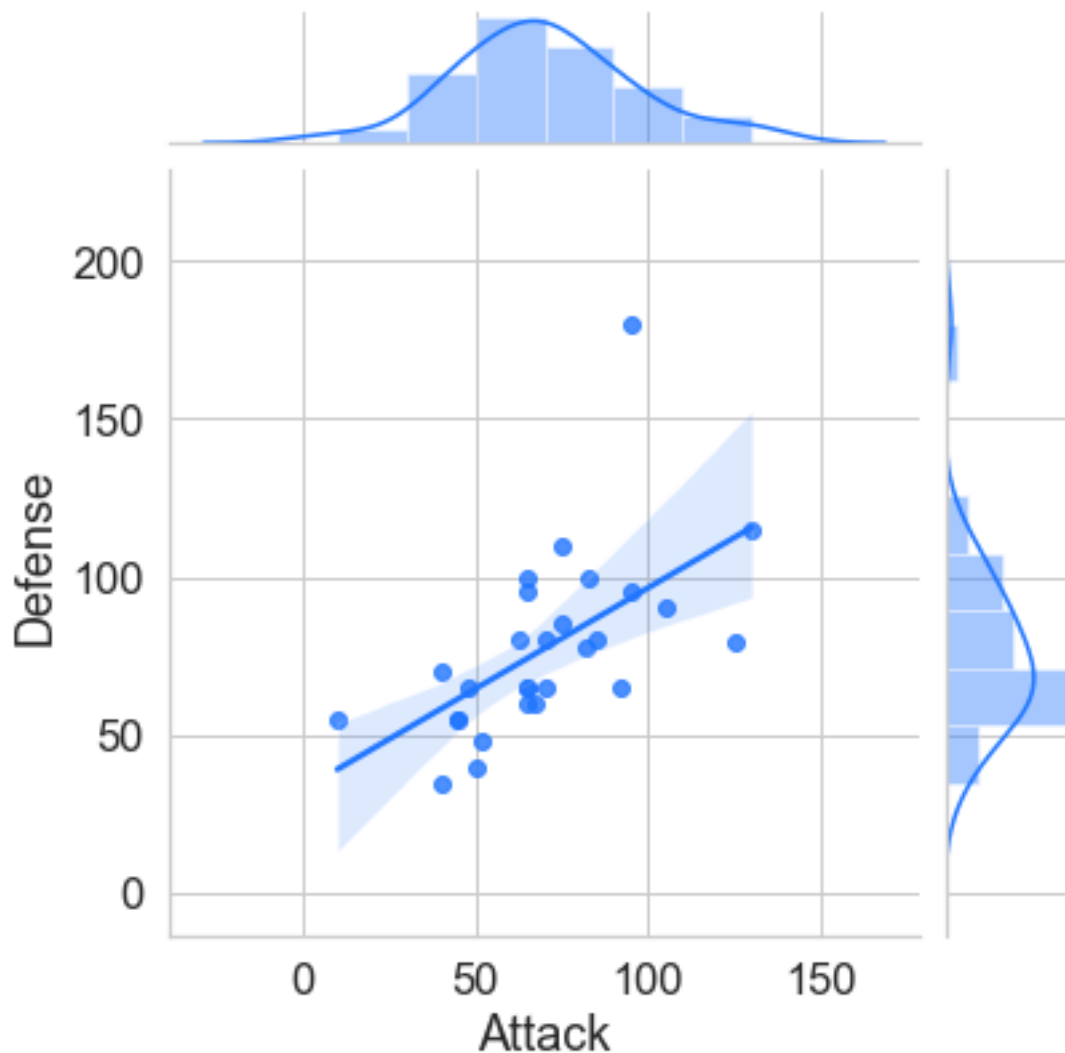
```
[14]: <seaborn.axisgrid.JointGrid at 0x1a20e23410>
```



```
[15]: #Indiquem que volem utilitzar l'estil "whitegrid" de seaborn
sns.set_style("whitegrid")

sns.jointplot(x='Attack', y='Defense',
             data=pkmn_inicials_tipus_stats[pkmn_inicials_tipus_stats['Type 1']=='Water'],
             color="#1f75fe", kind='reg')
```

```
[15]: <seaborn.axisgrid.JointGrid at 0x1a20ffbc50>
```



Resposta

No hi ha una sola solució correcta en aquest exercici ;) Una bona resposta seria recomanar un pokemon inicial d'aigua o foc, ja que tenen una correlació positiva entre atac i defensa, per tant, els pokemons evolucionats amb atac alt tindran també una defensa alta.

1.3.2 Exercici 2

Carregueu el conjunt de dades `pulitzer-circulation-data.csv` en un dataframe de Pandas. La font original d'aquest conjunt de dades és el repositori de dades de [FiveThirtyEight](#).

Genereu un diagrama de dispersió que permeti visualitzar les mostres del conjunt de dades de Pulitzer segons les variables `Daily Circulation 2004` i `Daily Circulation 2013`. Incloeu una recta amb l'ajust lineal entre les dues variables. Pel que fa als detalls de visualització, limiteu la visualització de tots dos eixos a l'interval (0, 1000000), feu servir l'estil `whitegrid` de Seaborn, i dibuixeu els punts i la línia en color verd.

Pista: podeu utilitzar la funció `jointplot` que hem vist al Notebook d'explicació. Penseu en quin tipus de gràfica, d'entre les que ofereix `jointplot`, s'ajusta als requeriments de l'enunciat.

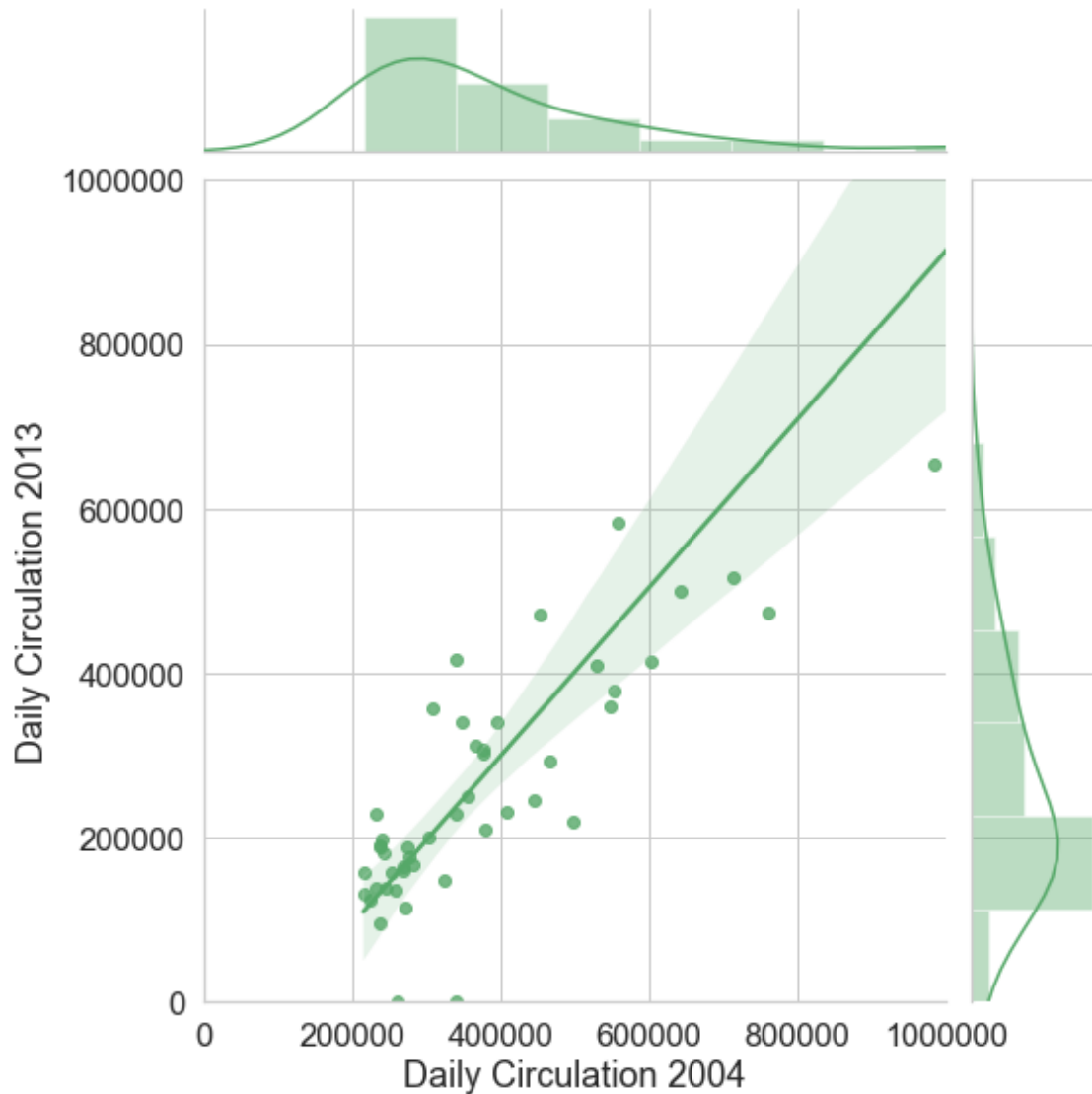
```
[16]: # Importem la llibreria seaborn
import pandas as pd
import seaborn as sns

# Mostrem les gràfiques en el notebook
%matplotlib inline

# Indiquem que volem utilitzar l'estil "whitegrid" de seaborn
sns.set_style("whitegrid")

# Carreguem les dades del fitxer pulitzer-circulation-data.csv en un dataframe
data = pd.read_csv('data/pulitzer-circulation-data.csv')

# Generem la gràfica de tipus "reg"
g = sns.jointplot("Daily Circulation 2004", "Daily Circulation 2013", data=data,
                  xlim=[0, 1000000], ylim=[0, 1000000],
                  kind="reg",
                  color="g", height=8)
```



1.3.3 Exercici 3

Joc de Trons és una serie de televisió basada en la famosa saga *Una Cançó de Foc i Gel* de George RR Martin. Aquesta serie és coneguda pels complicats escenaris polítics i bèlics, així com les nombroses morts de personatges.

En aquest exercici treballem amb el dataset `battles.csv` que ens dóna informació sobre les batalles que han tingut lloc al llarg de *Joc de Trons*.

Genereu un diagrama de punts (*scatter*) que permeti comparar la mida (`attacker_size` vs `defender_size`) dels dos principals exèrcits involucrats en cada batalla. Feu servir la columna `attacker_outcome` per mostrar amb diferents colors el resultat de la batalla per l'atacant.

Podeu utilitzar la funció `lmplot` de Seaborn per generar la gràfica. Penseu en quins atributs ens permeten crear aquest tipus de visualització.

Veient la gràfica, podem afirmar que la mida de l'exèrcit és determinant en l'*outcome* de la batalla?

Nota: feu servir l'estil `whitegrid` de Seaborn.

```
[17]: import pandas as pd
import seaborn as sns
%matplotlib inline

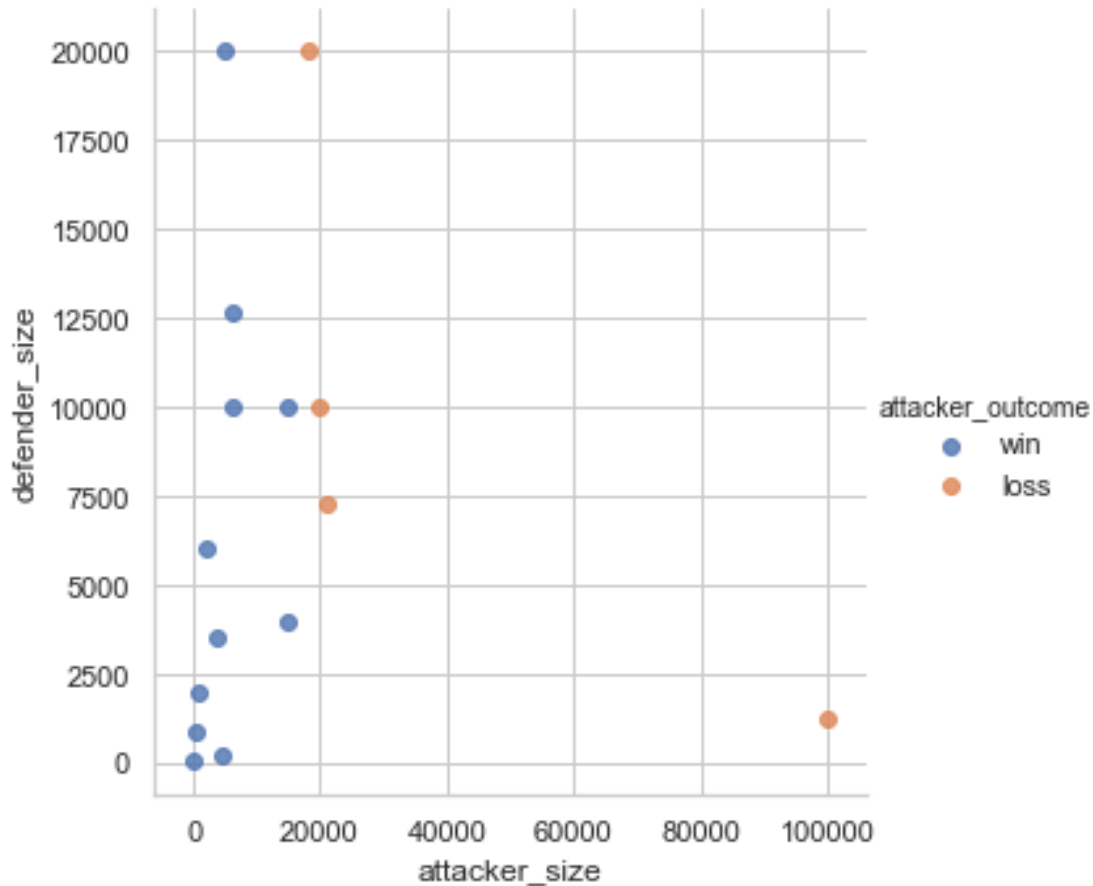
# Especifiquem que volem utilitzar l'estil whitegrid de Seaborn
sns.set(style="whitegrid")

# Carreguem les dades del fitxer got.csv en un dataframe
got_data = pd.read_csv('data/battles.csv')

# Eliminem les mostres amb valors NaN en les columnes attacker_size o
↳ defender_size
got_data = got_data.dropna(subset=["attacker_size", "defender_size"])

# Mostrem la gràfica
sns.lmplot(x="attacker_size", y="defender_size", hue="attacker_outcome",
↳ scatter=True, fit_reg=False, data=got_data)
```

```
[17]: <seaborn.axisgrid.FacetGrid at 0x1a1f2f4c10>
```



Resposta

Veient la gràfica anterior, no podem afirmar que la mida de l'exèrcit és determinant en l'outcome de la batalla. Com podem observar, hi ha casos on l'atacant perd la batalla tot i la superioritat numèrica de les tropes i a l'inrevés, l'atacant guanya la batalla tot i la superioritat numèrica del bàndol contrari.

1.3.4 Exercici 4

En els regnes de Ponent, els Grans Mestres consideren que tothom hauria de tenir accés a una biblioteca. Desgraciadament, després d'una gran guerra la xarxa de carreteres que comuniquen aquests regnes ha quedat greument afectada. Els Grans Mestres ens han contractat per resoldre les següents qüestions:

1. Quins regnes han quedat sense poder accedir a una biblioteca?
2. Quin és el nombre mínim de biblioteques que s'haurien de construir per tal de que tothom torni a tenir accés a una biblioteca? (si no podem re-construir cap carretera)
3. Quin és el cost mínim de re-construcció de carreteres per tal de que tothom torni a tenir accés a una biblioteca? (si no podem construir cap biblioteca)
4. Veient la resposta de la pregunta 2 i 3, què seria el més òptim tenint en compte que volem minimitzar els costos?

Per poder respondre a aquestes preguntes, els Grans Mestres ens faciliten la següent informació:

- Disposem d'un graf amb l'estat de la xarxa de carreteres entre els regnes.
- Disposem d'un llistat amb les ciutats que disposen d'una biblioteca.
- Una ciutat es considera que té accés a una biblioteca si existeix una biblioteca en la mateixa ciutat o si els habitants poden viatjar per carretera fins a una altra ciutat amb biblioteca.
- El cost de construir una biblioteca és de 150.000€.
- El cost de re-construir una carretera és de 80.000€.

Nota: per resoldre l'exercici, **heu de generar una visualització del graf que us permeti respondre a les quatre preguntes plantejades**. Després, recordeu respondre a les quatre preguntes!

```
[18]: # Dades disponibles per a l'exercici

import networkx as nx

# Llista de carreteres transitables
carreteres_ok = [(1,3), (3,4), (2, 7), (2, 8), (5, 6), (9, 10)]

# Llista de carreteres tallades
carreteres_ko = [(1, 2), (2, 4), (2, 5), (8, 9)]

# Graf G, que representa la xarxa de carreteres entre les ciutats
G = nx.Graph()

G.add_edges_from(carreteres_ok)
G.add_edges_from(carreteres_ko)

# Llista de ciutats amb biblioteca
ciutats_amb_biblioteca = [3, 6, 9]
```

```
[19]: %matplotlib inline

# Calculem la posició dels nodes utilitzant l'algoritme spring
graph_pos = nx.spring_layout(G)

# Mostrem els nodes del graf. Mostrem amb una mida més gran i en blau les
↳ ciutats que tenen biblioteca
nx.draw_networkx_nodes(G, graph_pos, ciutats_amb_biblioteca, node_size=700,
↳ node_color='blue', alpha = 0.6)

# Mostrem amb una mida més petita i en taronja els altres nodes del graf
ciutats_sense_biblioteca = list(set(G.nodes()) - set(ciutats_amb_biblioteca))
nx.draw_networkx_nodes(G, graph_pos, ciutats_sense_biblioteca, node_size=300,
↳ node_color='orange', alpha = 0.6)

# Mostrem les etiquetes del nodes, especificant el tamany i la font
```

```

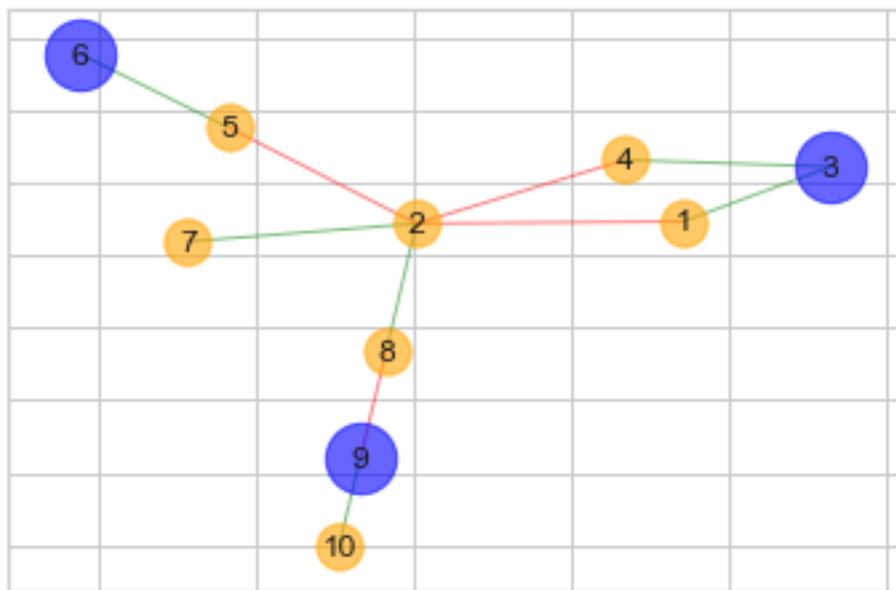
nx.draw_networkx_labels(G, graph_pos, font_size=12, font_family='sans-serif')

# Mostrem les aristes del graf
# Primer, mostrem amb color verd les aristes que corresponen a les carreteres
→ que encara són transitables
nx.draw_networkx_edges(G, graph_pos, edgelist=carreteres_ok,
→ edge_color='green', alpha=0.5)

# Mostrem amb color vermell les aristes que corresponen a les carreteres
→ tallades
nx.draw_networkx_edges(G, graph_pos, edgelist=carreteres_ko, edge_color='red',
→ alpha=0.5)

```

[19]: <matplotlib.collections.LineCollection at 0x1a21949f10>



Resposta

Una vegada construït el graf, podem contestar les preguntes que ens han fet els Grans Mestres.

- Quins regnes han quedat sense poder accedir a una biblioteca?

Els regnes que han quedat sense accés a una biblioteca són: [2, 7, 8].

- Quin és el nombre mínim de biblioteques que s'haurien de construir per tal de que tothom torni a tenir accés a una biblioteca? (si no podem re-construir cap carretera)

Donat que els tres regnes que han quedat sense accés a una biblioteca estan connectats entre sí (encara queden carreteres que els uneixen), podem contruir una biblioteca en qualsevol dels tres per tal de que els ciutadans d'aquests regnes puguin tornar a disfrutar d'una biblioteca. Per tant, el nombre mínim de biblioteques seria **1**.

- Quin és el cost mínim de re-construcció de carreteres per tal de que tothom torni a tenir accés a una biblioteca? (si no podem construir cap biblioteca)

Seguint el mateix raonament anterior, si re-construïm la carretera que connectava el regne 8 amb el regne 9 (que té biblioteca), també estarem habilitant l'accés als regnes 2 i 7. Per tant, el cost mínim per re-construir carreteres seria **150.000€**.

- Veient la resposta de la pregunta 2 i 3, què seria el més òptim tenint en compte que volem minimitzar els costos?

En aquest cas, el més òptim seria re-construir una carretera (per exemple la que uneix els regnes 8 i 9). Això costaria 80.000€, mentre que construir una biblioteca ens implica un cost de 150.000€.

1.3.5 Exercici 5

L'any 2018 a la ciutat de Barcelona s'han produït, de mitjana, 27.22 accidents amb vehicles implicats. És per això que l'ajuntament de Barcelona requereix del nostre servei per construir un mapa **interactiu** que mostri els punts on s'han produït els accidents.

Ens demanen crear un mapa utilitzant la llibreria *geoplotlib* que ens ha de permetre veure la localització de cada accident i el nombre de víctimes implicades. De moment només ens demanen visualitzar els accidents produïts al mes de **juny del 2018** en **caps de setmana** (dissabtes i diumenges).

Nota 1: Podeu obtenir les dades dels accidents gestionats per la Guàrdia Urbana a la ciutat de Barcelona al portal [Open Data BCN](#) i [carregar les dades](#) a partir d'un diccionari o un dataframe.

```
[20]: # Primer, ens descarreguem el fitxer csv amb les dades del portal de Open Data
      ↪BCN
      # (https://opendata-ajuntament.barcelona.cat/data/ca/dataset/accidents-gu-bcn)
      # i el guardem a la carpeta data per poder carregar-lo a posteriori
      import pandas as pd
      import geoplotlib

      # Carreguem les dades en un dataframe
      accidents_data = pd.read_csv('data/2018_accidents_gu_bcn.csv')

      # Inspeccionem les dades que hem carregat
      accidents_data.info(verbose=True)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 9936 entries, 0 to 9935
```

```
Data columns (total 27 columns):
```

#	Column	Non-Null Count	Dtype
0	Numero_expedient	9936 non-null	object
1	Codi_districte	9936 non-null	int64
2	Nom_districte	9936 non-null	object
3	Codi_barri	9936 non-null	int64
4	Nom_barri	9936 non-null	object
5	Codi_carrer	9936 non-null	int64

```

6   Nom_carrer          9935 non-null   object
7   Num_postal          5138 non-null   object
8   Descripcio_dia_setmana 9936 non-null   object
9   Dia_setmana         9936 non-null   object
10  Descripcio_tipus_dia  9936 non-null   object
11  Any                 9936 non-null   int64
12  Mes_any            9936 non-null   int64
13  Nom_mes            9936 non-null   object
14  Dia_mes            9936 non-null   int64
15  Hora_dia           9936 non-null   int64
16  Descripcio_torn     9936 non-null   object
17  Descripcio_causa_vianant 9936 non-null   object
18  Numero_morts        9936 non-null   int64
19  Numero_lesionats_lleus 9936 non-null   int64
20  Numero_lesionats_greus 9936 non-null   int64
21  Numero_victimes     9936 non-null   int64
22  Numero_vehicles_implicats 9936 non-null   int64
23  Coordenada_UTM_X     9936 non-null   float64
24  Coordenada_UTM_Y     9936 non-null   float64
25  Longitud            9936 non-null   float64
26  Latitud             9936 non-null   float64
dtypes: float64(4), int64(12), object(11)
memory usage: 2.0+ MB

```

```

[21]: # Inspeccionem els valors de la columna Mes_any i Dia_setmana
# ja que ens demanen filtrar pel mes de juny i caps de setmana
print(accidents_data['Mes_any'].unique())
print(accidents_data['Dia_setmana'].unique())

```

```

[ 1  7  2  9  3 10 12  5  4  6 11  8]
['Dl' 'Dg' 'Dc' 'Dv' 'Dj' 'Ds' 'Dm']

```

```

[22]: # Geoplotlib necessita dues columnes amb el nom lat i lon per poder dibuixar
      ↪ els punts sobre el mapa.
# Renombrem les columnes Longitud i Latitud
accidents_data.rename(index=str, columns={'Longitud': 'lon', 'Latitud': 'lat'},
      ↪ inplace=True)

```

```

[23]: # Filbrem per les dades corresponents al mes de juny 2018 i caps de setmana
accidents_juny = accidents_data.loc[(accidents_data['Mes_any'] == 6)
      & (accidents_data['Dia_setmana'].
      ↪ isin(['Ds', 'Dg']))]

# Carreguem amb geoplotlib les dades del dataframe
gdata = geoplotlib.utils.DataAccessObject.from_dataframe(accidents_juny)

# Mostrem els punts del fitxer

```

```
geoplotlib.dot(gdata, color='b', point_size=1.75, f_tooltip=lambda x :  
↳str(x['Numero_victimes'])) )  
  
# Mostrem el mapa  
geoplotlib.show()
```

1.3.6 Exercici 6

En aquest exercici utilitzarem el *dataset* `covid19_US_cases.csv`, que mostra l'evolució dels casos de Covid-19 als diferents estats des de finals de gener del 2020.

1. Quina és la distribució de casos als Estats Units l'últim mes (abril)? **Mostra gràficament els resultats per cada grup (actius, recuperats i morts), però sense separar per estats.**
2. Addicionalment, se'ns demana generar una gràfica que mostri l'evolució dels casos confirmats a l'estat de Nova York, Massachusetts, Washington, California i Texas des de l'inici de la pandèmia. Quina regió és la més afectada?

Nota 1: En l'apartat 1, per sumar tots els casos de US es pot utilitzar la funció `groupby` de `pandas` que hem vist en lliçons anteriors.

Nota 2: La visualització de les dades hauria de ser atractiva i clara per tal de que es puguin treure conclusions d'elles.

```
[3]: import numpy as np  
import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
# Mostrem les gràfiques en el notebook  
%matplotlib inline  
  
# Indiquem que volem utilitzar l'estil "whitegrid" de seaborn  
sns.set_style("whitegrid")  
  
# Carreguem les dades del fitxer covid19_US.csv en un dataframe  
covid_US_cases = pd.read_csv('data/covid19_US_cases.csv')  
  
# Visualitzem les primeres files  
covid_US_cases.head()
```

```
[3]:
```

	ObservationDate	Province/State	Country/Region	Confirmed	Deaths	Recovered	\
0	03/13/2020	Alabama	US	5	0	0	
1	03/14/2020	Alabama	US	6	0	0	
2	03/15/2020	Alabama	US	12	0	0	
3	03/16/2020	Alabama	US	29	0	0	
4	03/17/2020	Alabama	US	39	0	0	

	Active
0	5
1	6
2	12
3	29
4	39

```
[4]: ##1

# Quina és la distribució de casos als Estats Units?

# Agrupem les dades per data i sumem els casos.

covid_US_all_cases=covid_US_cases.groupby('ObservationDate').sum()

# Creem dues columnes noves amb la informació del mes i del dia de l'observació
covid_US_all_cases['Month']=pd.DatetimeIndex(covid_US_all_cases.index).month
covid_US_all_cases['Day']=pd.DatetimeIndex(covid_US_all_cases.index).day

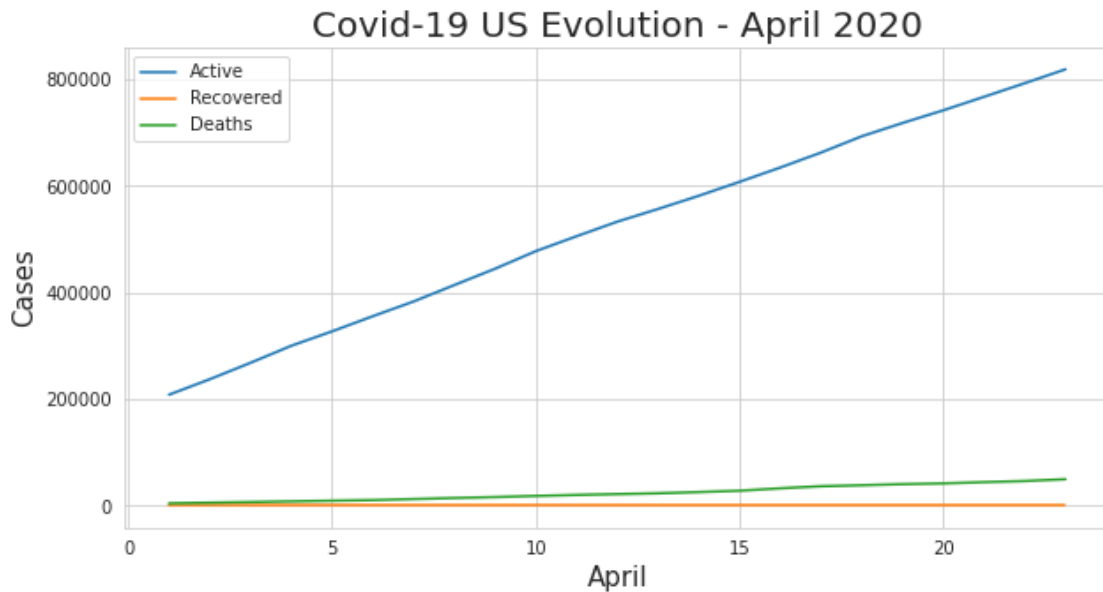
# Filtrem el dataset per agafar només les observacions d'abril
covid_US_all_cases_april=covid_US_all_cases[covid_US_all_cases['Month']==4]
```

```
[5]: # Grafiquem el resultat

plt.figure(figsize = (10,5))

ax = sns.lineplot(x='Day', y="Active", data=covid_US_all_cases_april)
ax = sns.lineplot(x='Day', y="Recovered", data=covid_US_all_cases_april)
ax = sns.lineplot(x='Day', y="Deaths", data=covid_US_all_cases_april)

plt.legend(loc='upper left', labels=['Active', 'Recovered', 'Deaths'])
plt.title("Covid-19 US Evolution - April 2020", fontsize = 20)
plt.xlabel("April", fontsize = 15)
plt.ylabel("Cases", fontsize = 15)
plt.show()
```

[6]: ##2

```
# Evolució dels casos confirmats a l'estat de Nova York, Massachusetts,
↳ Washington, California i Texas

# Fem una llista amb els estats que volem estudiar
list_subset=['New York','Massachusetts','Washington','California','Texas']

# Agafem el subset amb aquests estats
covid_US_cases_subset=covid_US_cases[covid_US_cases['Province/State']
↳ isin(list_subset)]

# Especifiquem el tamany i el format del gràfic
plt.figure(figsize = (14,7))
sns.set_style("whitegrid", {'axes.grid' : False})

# Fem el gràfic indicant que els resultats es mostraran en funció de l'estat
sns.lineplot(x = "ObservationDate", y = "Confirmed", data =_
↳ covid_US_cases_subset, hue = "Province/State",
style = "Province/State", dashes = False,
markers = ["o", "<", ">", "^", "*"], legend="brief")

# Inclinem les dates perquè es puguin llegir bé
plt.xticks(
rotation=45,
```

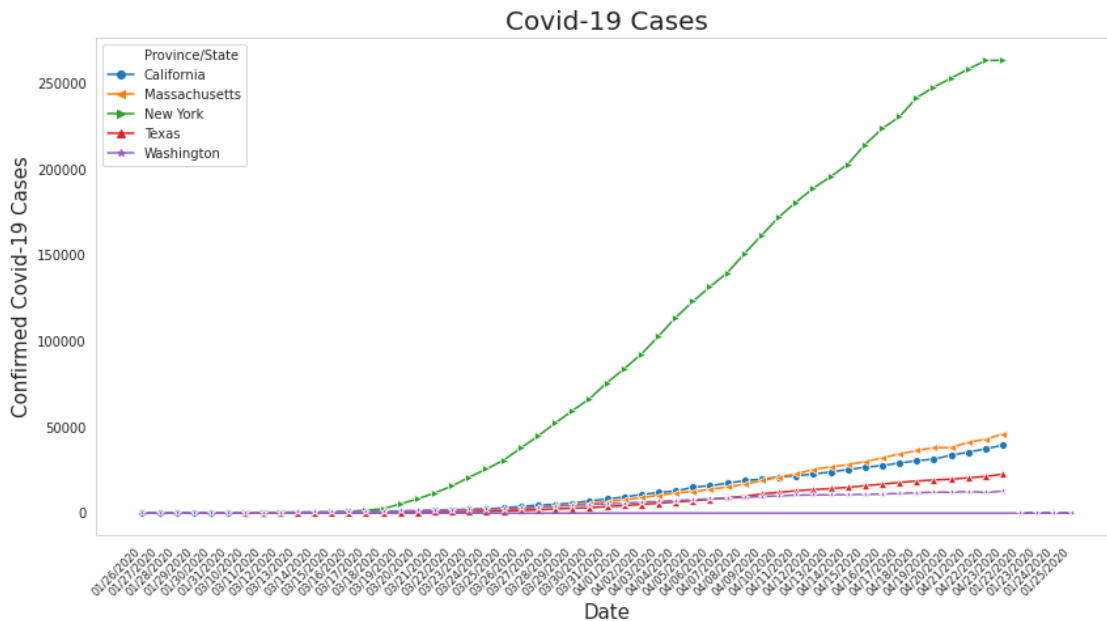
```

        horizontalalignment='right',
        fontweight='light',
        fontsize='small'
    )

plt.title("Covid-19 Cases", fontsize = 20)
plt.xlabel("Date", fontsize = 15)
plt.ylabel("Confirmed Covid-19 Cases", fontsize = 15)

plt.show()

```



Resposta

Amb aquest gràfic es pot veure clarament que l'estat més afectat pel Covid-19 és Nova York.

1.3.7 Exercici 7

A partir de les dades de twitter es poden crear xarxes de relacions entre els seus usuaris. En aquest exercici treballarem amb el conjunt de dades `politics_twitter.csv`, que constitueix una xarxa de les relacions entre diferents polítics de l'estat espanyol generada a partir d'extraccions de twitter.

Cada fila representa un link entre 2 polítics (P1 i P2) i el valor (Weight) indica la força de la connexió, en aquest cas 1 ja que només fa referència a la presència d'aquesta relació.

1. Quantes comunitats es poden detectar al graf?
2. Representa gràficament el graf i pinta de diferent color cada comunitat detectada.

```
[7]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx

# Carreguem les dades del fitxer book1.csv en un dataframe
twitter = pd.read_csv('data/politics_twitter.csv')

# Visualitzem les primeres files del dataset
twitter.head()
```

```
[7]:
```

	P1	P2	Weight
0	KRLS	IreneMontero	1
1	LauraBorras	IreneMontero	1
2	gabrielrufian	IreneMontero	1
3	AdaColau	IreneMontero	1
4	Jaumeasens	IreneMontero	1

```
[8]: # Dades disponibles per a l'exercici

# Creem un objecte graf buit
G_twitter= nx.Graph()

# Iterem per totes les files del dataset
for _, edge in twitter.iterrows():
    G_twitter.add_edge(edge['P1'], edge['P2'], weight=edge['Weight'])
```

```
[9]: ## 1

# Quantes comunitats poden detectar en el graf?

# Importem la llibreria de detecció de comunitats
from community import community_louvain

# Detectem les comunitats existents al graf
# N'hi ha 3
partition = community_louvain.best_partition(G_twitter)
```

```
[10]: ## 2

# Representa gràficament el graf i pinta de diferent color cada comunitat_
↳ detectada

# Importem la llibreria de detecció de comunitats
from community import community_louvain

# Generem una nova figura
```

```

plt.figure(1, figsize=(12, 10))

# Calculem les posicions dels nodes del graf en el plànol amb l'algoritme spring
graph_pos = nx.spring_layout(G_twitter)

# Definim els colors que utilitzarem per pintar les 3 comunitats
colors = ['green', 'blue', 'red']

# Per a cada comunitat detectada, en mostrem els nodes
for count, com in enumerate(set(partition.values())):

    # Seleccionem els nodes de la comunitat que estem processant
    list_nodes = [nodes for nodes in partition.keys()
                   if partition[nodes] == com]

    # Mostrem els nodes, acolorits segons la comunitat a la qual pertanyen
    nx.draw_networkx_nodes(G_twitter, graph_pos, list_nodes,
                           node_size = 800,
                           node_color = colors[count],
                           alpha = 0.5)

# Mostrem les arestes del graf, especificant-ne la posició,
# el color i la transparència
nx.draw_networkx_edges(G_twitter, graph_pos, edge_color='k', alpha=0.3)

# Mostrem les etiquetes, indicant-ne la font i la grandària
a = nx.draw_networkx_labels(G_twitter, graph_pos, font_size=12,
                             font_family='sans-serif')

```

